

1- Why should one use Azure Key Vault when working in the Azure environment? What are the alternatives to using Azure Key Vault? What are the pros and cons of using Azure Key Vault?

- Azure Key Vault can be used to Securely store and tightly control access to tokens, passwords, certificates, API keys, and other secrets
- Azure Key Vault can be used as a key management solution. Azure Key Vault makes it easy to create and control the encryption keys used to encrypt your data.
- Azure Key Vault lets you easily provision, manage, and deploy public and private Transport Layer Security/Secure Sockets Layer (TLS/SSL) certificates for use with Azure and your internal connected resources.
- Azure Key Vault has two service tiers: Standard, which encrypts with a software key, and a Premium tier, which includes hardware security module (HSM)-protected keys.

Azure alternatives:

1. Google Cloud Key Management Service (KMS)
2. AWS Secrets Manager

Advantages

Data security is maximized because Key Vault provides a centralized location for your sensitive information.

it has a seamless integration with azure services and applications

It can maintain different versions of the password without the need to recreate a new line item in the key vault.

Key Vault is a scalable service. This means it does not face any throttling if multiple services try to access the secrets and the volume of requests increases.

Disadvantages

It's not free. It's a paid service and the cost is formed on a variety of factors such as number of keys and secrets stored, number of operations performed, etc.

A bit of a learning curve is involved if your team has not worked with Key Vault before.

Using key vault introduces dependency on Azure

2- How do you achieve the loop functionality within an Azure Data Factory pipeline? Why would you need to use this functionality in a data pipeline?

Loop functionality in an Azure Data Factory (ADF) pipeline is essential when data workflow needs to **handle repetitive or dynamic tasks**. The **ForEach** activity defines a repeating

control flow in an Azure Data Factory or Synapse pipeline. This activity is used to iterate over a collection and execute specified activities in a loop.

- 3- What are expressions in Azure Data Factory? How are they helpful when designing a data pipeline (please explain with an example)?

In Azure Data Factory (ADF), expressions are a powerful language used to define dynamic behavior and logic within pipelines and data flows.

1. **Functions and System Variables:** Predefined variables that provide information about the current pipeline execution (e.g., `pipeline().RunId`, `activity().Name`).
2. **Dynamic Content:** To configure properties like file paths, connection strings, and activity inputs based on dynamic values. **Example:** Dynamic File Naming in Copy Activity Sink "`@concat('processed_data_', formatDateTime(utcNow(), 'yyyyMMdd'), '.csv')`"
3. **Conditional Logic:** In activities like If Condition or Switch, to control pipeline flow based on evaluated expressions. **Example** —Conditional Logic in an If Condition Activity "`@if(equals(variables('status'), 'success'), 'Proceed', 'Retry')`"
4. **Data Transformation:** Expressions play a crucial role in transforming data during activities like DerivedColumn, where you can modify schema or values based on defined rules or conditions.
5. **Looping and Iteration:** Iterative activities like ForEach and Until rely heavily on expressions (`@foreach`, `@concat`, etc.) to define loop conditions, iterate over collections, and dynamically generate activity configurations.
6. **Data Validation and Error Handling:** Expressions can be used to validate data (`@empty`, `@contains`, etc.) before processing or to handle errors during execution, ensuring robust data pipelines. **Example:** Before copying a file, check **if it exists** (i.e., the file list is not empty). `@not(empty(activity('Get Metadata').output.childItems))`

- 4- What are the pros and cons of parametrizing a dataset in Azure Data Factory pipeline's activity?

Parameters can be defined at several levels, including the activity, pipeline, and dataset levels. Parameterizing a dataset in an **Azure Data Factory (ADF)** pipeline activity means making the dataset **dynamic**—so it can accept **values at runtime** (like file names, table names, paths, dates, etc.).

Pros:

1. **Reusability:** use **one dataset** for multiple sources (e.g., different files, folders, or tables).
2. **Flexibility:** Design dynamic pipelines that can handle different inputs by passing parameter values at runtime

Example: `@concat('input/', pipeline().parameters.date, '.csv')` for dynamic file input.

3. **Maintainability:** minimizes the amount of hard coding and increases the number of reusable objects and processes in a solution.
4. **Scalability:** Build scalable solutions by making pipelines more adaptable to changing requirements. Ideal for **ForEach loops**, where the dataset can change per iteration using `@item()` values.

Cons:

1. Increased Complexity
 2. **Debugging is Harder:** Troubleshooting can become more challenging if there are issues with incorrect parameter values
-
- 5- What are the different supported file formats and compression codecs in Azure Data Factory? When will you use a Parquet file over an ORC file? Why would you choose an AVRO file format over a Parquet file format?
 - ✓ Avro
 - ✓ Binary
 - ✓ Delimited text
 - ✓ Excel
 - ✓ JSON
 - ✓ ORC
 - ✓ XML
 - ✓ Parquet
-
- **Parquet**, originally developed by Twitter and now part of the Apache ecosystem, is known for its broad compatibility with big data tools such as Spark, Hive, Impala, and Drill. Parquet uses a hybrid encoding scheme and supports complex nested data structures, making it ideal for read-heavy workloads and analytics across various platforms. Parquet also provides excellent compression and efficient I/O by enabling selective column reads, reducing the amount of data scanned during queries.
 - **ORC** was specifically designed for **Hadoop** ecosystem and optimized for **Hive**. It generally offers better compression ratios and better read performance for certain types of queries due to its lightweight indexing and aggressive predicate pushdown capabilities. ORC includes built-in statistics and supports lightweight indexes, which can accelerate filtering operations significantly. While Parquet offers broader tool compatibility, ORC often outperforms it within Hive-centric environments, especially when dealing with flat data structures and large sequential scans.
 - **Avro** file format is usually used in streaming scenarios for its serialization and schema handling capabilities and for its seamless integration with Apache Kafka, offering a powerful combination for handling real-time data streams. Avro, in contrast with Parquet and ORC, is a row-based storage format designed for efficient data serialization and schema evolution. Avro excels in write-heavy use cases like data ingestion and streaming and is commonly used with Kafka. Unlike Parquet and ORC, which are optimized for analytical queries, Avro is designed for fast reads and writes of complete records, and it stores the schema alongside the data, enabling easier data exchange and evolution over time.

Reference for the response: (<https://aws.amazon.com/blogs/big-data/compaction-support-for-avro-and-orc-file-formats-in-apache-iceberg-tables-in-amazon-s3/>)